Lexical Task Heads beyond Simple Tasks: Complex Rule-Defined Tasks

Filippo Merlo

Draft — November 3, 2026

Abstract

Lexical task heads are attention heads whose output, decoded into the vocabulary, names the task the prompt asks for (Yang et al., 2026). They were identified with the per-head logit lens on simple single-step tasks, such as mapping a country to its capital. We ask whether the same method finds them for complex tasks defined by explicit rules (Fu et al., 2026), which need several inference steps, and whether Head-JLens (Merlo, 2026), which decodes each head's output after mapping it through a Jacobian of the remaining layers averaged over many texts, finds lexical task heads on them that the logit lens misses. In Llama-3.1-8B-Instruct, we identify lexical task heads under both readouts on 5 complex tasks at their easy level, with rule-based, example-based and combined prompts, and test each head set by activation patching. The identification method and its results on simple tasks are given in a companion document.

1 Introduction

A language model's accuracy on a task can change a great deal with the wording of the prompt that asks for it: on Llama-3.1-8B-Instruct, prompts for the same antonym items differ in accuracy by up to a factor of four (Yang et al., 2026, Fig. 1). Yang et al. (2026) explain part of this variation with *lexical task heads*, attention heads whose outputs, decoded into the vocabulary with the logit lens, name the task ("opposite" for antonyms, "cities" for capitals). The heads are largely shared between instruction-based and example-based prompts, patching their mean activation into a prompt without task information raises task accuracy, and prompts that fail activate them less.

Two limits of that work motivate this project. First, its 17 tasks are simple single-step mappings (country to capital, word to antonym), and the authors note that "it remains unclear if these shared lexical mechanisms hold for more abstract tasks or tasks that are less easily defined by simple vocabulary clusters" (Yang et al., 2026, Sec. 4). Fu et al. (2026) provide such tasks: five novel tasks defined by explicit rules (a card game, a board game, a composed arithmetic operator, noun-class agreement in an artificial language, and lexical-category inference), each at three difficulty levels and each given as rules, as examples or as both. Second, the logit lens decodes a head's output directly and leaves out the computation of the layers that follow it; Yang et al. (2026, Sec. 4) note that task representations "encoded in a non-lexical, abstract format" may not project cleanly to tokens. Head-JLens reads each head through a Jacobian map of the remaining layers, and on Llama-3.1-8B-Instruct it ranks annotated intermediate words, which the prompt never states (such as the bridge entity of a two-hop question), higher than the per-head logit lens at most layers on five of six evaluation sets (Merlo, 2026, Sec. 3).

We therefore ask whether Yang et al.'s method identifies lexical task heads for the complex tasks of Fu et al. (2026), and how the logit lens and Head-JLens compare as readouts for identifying them. The method is worked out on the simple tasks in a companion document (`proposal-simple.tex`); this document uses its methods and compares with its results.

Contributions.

1. A test of the lexical-task-head method on complex, rule-defined tasks, with a causal check of the identified heads.
2. A comparison of the per-head logit lens and Head-JLens for identifying lexical task heads on these tasks.

Scope. We do not explain behavioural variation across wordings (the failure analyses of Yang et al., 2026, Sec. 2.4), and we use one model.

2 Related work

Task representations across prompting styles. Function vectors are sums of the mean outputs of a few attention heads that, added to the residual stream, trigger a task in another context (Todd et al., 2024). Davidson et al. (2025) extract them from instructions as well as from demonstrations and find that the two prompting styles engage mostly different heads: of the 20 top heads in each model, 2 to 7 are shared. Yang et al. (2026) identify a different kind of task representation, lexical task heads, whose outputs decode to words describing the task; 73% of them are shared between instruction-based and example-based prompts on average, and they are largely disjoint from function-vector heads (Yang et al., 2026, App. I). We test the lexical-task-head method on complex tasks.

Reading attention heads in the vocabulary. The logit lens decodes an intermediate state with the model's final normalization and unembedding (nostalgebraist, 2020); applied to one head's output, it is the readout of Yang et al. (2026). The Jacobian lens of Gurnee et al. (2026) instead maps a residual-stream state through a Jacobian of the remaining layers, averaged over many texts. Head-JLens moves this map to the concatenated head outputs, so that each head is read through the layers that follow it; in Llama-3.1-8B-Instruct it ranks annotated intermediate words higher than the per-head logit lens at most layers on five of six evaluation sets, and heads selected with it are better sites for edits (Merlo, 2026). We compare the two readouts on a different use: identifying task-descriptive heads.

Learning tasks from rules and from examples. Fu et al. (2026) compare learning from rules, from examples and from both on five novel tasks. Rules outperform examples overall, adding examples to rules gives no significant gain, and the advantage of rules is largest for tasks that need algebraic computation. Their study is behavioural; we use their tasks to study task representations.

3 Research questions

RQ1 Does the method of Yang et al. (2026) identify lexical task heads for the complex tasks of Fu et al. (2026) under rule-based and example-based prompts, and do the identified heads have a causal effect on task accuracy?

RQ2 How do the two readouts compare for identifying lexical task heads on the complex tasks?

RQ3 Are the lexical task heads of a complex task shared between rule-based and example-based prompts, as Yang et al. (2026) find for simple tasks?

Hypothesis 1 (complex tasks)**.** *Following the limitation stated by Yang et al. (2026, Sec. 4), the complex tasks yield fewer lexical task heads per task and a smaller causal effect than the simple tasks under Logit-head.* Falsified *if they yield as many heads with as large an effect.*

Hypothesis 2 (readouts)**.** *Following Merlo (2026), on the complex tasks J-head identifies at least the lexical task heads that Logit-head identifies, and the heads found only by J-head have a causal effect above baseline.* Falsified *if the J-head set misses heads of the Logit-head set, or if the heads found only by J-head have no causal effect.*

4 Framework

Tasks and prompts. A task t is given by a dataset D_t of inputs and correct outputs. For the complex tasks we use the easy level of each (Sec. 5.2). Each prompt for t has a style $s \in \{\text{rule}, \text{example}, \text{combined}\}$: its task information is a rule text, a set of demonstrations, or both, followed by the input x. $P_{t,s}$ is the set of prompts of style s for t, and $P^{+}_{t,s} \subseteq P_{t,s}$ the prompts whose heads are read: those the model answers correctly under the flexible score of Sec. 5.5, for both families of tasks, whether or not the answer is the first generated text.

Heads and readouts. Following Merlo (2026, Sec. 2), $z_{l,h}(p)$ is the output of head h in layer l at the last position of p, before the output projection, and $c_{l,h}(p) = W^{(h)}_{O,l} z_{l,h}(p)$ its write to the residual stream. With $q_v = g \odot w_v$ the gain-scaled unembedding row of token v, the two readouts score v by

$$\ell^{\mathrm{L}}_{l,h}(p)[v] = q_v^{\top} c_{l,h}(p) \quad \text{(Logit-head)}, \qquad \ell^{\mathrm{J}}_{l,h}(p)[v] = q_v^{\top} J_{l,h}\, z_{l,h}(p) \quad \text{(J-head)}.$$

$T^{R}_{k,l,h}(p)$ is the set of the k highest-scoring tokens under readout $R \in \{\mathrm{L}, \mathrm{J}\}$, and V_t the term list of task t.

Definition 1 (Lexical task head)**.** *Following Yang et al. (2026, Sec. 2.2.2), head (l, h) is a lexical task head for t on prompt p under readout R if $|T^{R}_{k,l,h}(p) \cap V_t| \geq n$. It is a lexical task head for t and style s if this holds for at least a fraction ρ of the prompts in $P^{+}_{t,s}$. We use $n = 1$, $k = 10$, $\rho = 0.1$, as Yang et al. (2026). $H^{R}_{t,s}$ denotes the set of such heads.*

5 Methods

5.1 Model

Llama-3.1-8B-Instruct: it is the main model of Yang et al. (2026), and the only model with released Head-JLens matrices.

5.2 Tasks

Simple tasks. The 14 tasks used by both Yang et al. (2026) and Davidson et al. (2025), with the datasets of Todd et al. (2024): antonym, synonym, singular–plural, present–past, country–capital, country–currency, landmark–country, park–country, product–company, next-item, prev-item, English–French, English–German, English–Spanish. Rule-based prompts: the five instruction templates per task of Yang et al. (2026, App. A.3.1). Example-based prompts: 1, 5, 10, 20 and 30 shots (Yang et al., 2026, App. A.3).

Complex tasks. The five tasks of Fu et al. (2026) at their easy level: Set Game, Tapatan, Operator Function, Noun Class Agreement, Lexical Category Inference. Data from their public release: 300 test items per task, drawn with their sampler and seed (all 600 for Lexical Category Inference), the same items under every prompt. Rule-based prompts: their rules-only prompts (Fu et al., 2026, App. F) and further wordings (Sec. 5.3). Example-based and combined prompts: their minimum-coverage demonstration sets (Fu et al., 2026, Sec. 2.5).

Why these tasks. Lexical task heads and their term lists were defined on the simple tasks (Yang et al., 2026), so the method applies unchanged and our results can be compared with theirs. Yang et al. (2026, Sec. 4) restrict their study to tasks with simple lexical descriptions. The complex tasks are novel, need several inference steps, and are given in rule, example and combined forms that specify the same task on the same test items, so the task is held fixed while the way it is communicated varies (Fu et al., 2026, Sec. 1–2).

Answer format. As in the source studies: the templates of Yang et al. (2026) for the simple tasks, and the answer formats of Fu et al. (2026, App. F) for the complex tasks, under the chat template. Decoding is greedy, as in Fu et al. (2026, App. A) (temperature 0; their released code defaults to 0.1), so that the prompt whose heads are read and the scored answer always match. The five complex tasks are generated alike (Sec. 5.3): the model may explain or reason before it answers, as Fu et al.'s instruction-tuned models often do. Lexical task heads are read at the last prompt token, before any generated text, for every task.

5.3 Task-representation methods

Lexical task heads (Yang et al., 2026, Sec. 2.2.2). Each attention head's output at the last prompt token is decoded with the two readouts of Sec. 4: Logit-head, the per-head logit lens used by Yang et al. (2026) (nostalgebraist, 2020), and J-head (Head-JLens), which applies the head's columns $J_{l,h}$ of the per-layer Jacobian map fitted on WikiText-103 (Merlo, 2026, Sec. 2). Fitted matrices for Llama-3.1-8B-Instruct are released with Merlo (2026, Sec. S4). Under each readout, lexical task heads are identified with Definition 1. For the simple tasks, $P^+_{t,s}$ is taken as in the code of Yang et al. (2026): the first 100 prompts in P^+ under one wording, the 5-shot prompts for the example style and the instruction template with the highest accuracy for the rule style. The complex tasks follow the same rule: the first 100 such prompts under the most accurate rule wording, under the example-based prompts and under the combined prompts. Term lists: Yang et al. (2026, App. A.2.1) for the simple tasks; for the complex tasks, see below.

Simple tasks as in the source code. The simple tasks are run as in the code of Yang et al. (2026), with three departures. First, padding: their code passes prompts to the model in batches of 20, padded on the left, without an attention mask, so that the model attends to the padding; we pass the mask and the matching positions, so that each prompt is computed as if run alone. Second, the J-head readout. Third, the precision of the forward pass: bfloat16, in which the Head-JLens matrices were fitted.

What we add to the complex tasks. Fu et al. (2026) provide the data, one rules-only prompt per task and difficulty level, and the example-based and combined prompts. To apply lexical task heads to them we add three things.

1. *Rule wordings*: six per original rule prompt (App. A), each stating exactly the same rules. They are generated with GPT-5.5 through the Azure OpenAI API from a paraphrase prompt that keeps placeholders, output labels and the answer-format instruction verbatim (App. B), and each is checked by us against the original; App. C lists them. Yang et al. (2026) use five instruction templates per simple task; the complex tasks have six rule wordings besides the original, and their rule-style P^+ is taken from the most accurate of the seven.

2. *Term lists* of task-descriptive words, of the kind in Yang et al. (2026, App. A.2.1): single words that describe the task rather than its answers, with case and number variants. They are generated with GPT-5.5 through the Azure OpenAI API from the rule prompt (App. B), as Yang et al. (2026, App. O) generated term lists with LLMs, and checked

by us. Each word is expanded to token ids as in Yang et al. (2026): as written, with a leading space, in upper, capitalised and lower case, and after a `.`, `_` or `-`, keeping every variant that is a single token of the model's tokenizer. There is one list per original rule prompt (App. A): Tapatan's levels share one prompt and change only board size, piece count and line length, whereas the other tasks add rules or, for Operator Function, change the function (Fu et al., 2026, Table 1, App. F). Lists are task-specific, not item-specific: for Lexical Category Inference, whose category definitions change from item to item, the list contains only words that describe the task itself. App. D lists them.

3. *Prompts without task information* for the causal check (Sec. 5.4): the input block of the rules-only prompt alone (the lines that introduce the input and the answer, such as `Sentence: {sentence}` and `Answer:`), without rules or examples, the counterpart of the zero-shot prompts of Yang et al. (2026, Sec. 2.3.2).

Generation and scoring of the complex tasks. All five complex tasks are run in one way: chat template, greedy decoding, and a reply that ends at the end-of-sequence token or at 1,024 new tokens. Fu et al. give instruction-tuned models longer limits (6,000 new tokens for Operator Function, App. A). In an earlier run with that limit on Set Game, Operator Function and Tapatan (ten prompt variants, 300 items), 4 of 9,000 replies reached a correct answer after token 1,024, and 690 of the 804 longer replies ran to 6,000 tokens without one. The strict score is the evaluation of Fu et al. (2026) on the whole reply: for Set Game and Operator Function, after whitespace is collapsed, the first `\boxed{}`, read as three cards of the form `(animal | biome | ...)` or as an integer; for Noun Class Agreement, the first line that is only Yes or No (after an optional `Answer:`), else a line that begins `Answer:` or `Final answer:` followed by Yes or No, else the first Yes or No anywhere in the reply; for Lexical Category Inference, the reply's first word, which must be Yes or No. Their code has no scorer for Tapatan (it stores the reply only), and their paper describes none; its prompt asks for the label alone. We read the first of its three labels (`A win`, `B win`, `continue`) anywhere in the reply, as in our first run of E0. A reply without an extractable answer is wrong. Where items share a prompt (Tapatan's sample repeats 46 boards, Noun Class Agreement's 52 sentences), the prompt is answered once and its reply scored for each of them.

Why these readouts. Lexical task heads locate a task representation and give its content in words (Yang et al., 2026). Logit-head reproduces their readout. J-head reads each head through the layers that follow it: in Llama-3.1-8B-Instruct it ranks unstated intermediate words higher than Logit-head at most layers on five of six evaluation sets, and its readouts are better guides for edits (Merlo, 2026, Secs. 3–4). A lexical task head's decoded task word is likewise not the answer, and Yang et al. (2026, Sec. 4) note that their logit-lens criterion may miss task representations that do not project cleanly to tokens.

5.4 Causal check

The causal check of Yang et al. (2026, Sec. 2.3.2, App. G) tests whether a head set carries the task: the heads' mean output on task prompts is written into prompts that do not state the task, and we measure whether the model then performs it.

1. *Patched prompts*: prompts without task information, the zero-shot prompts of Yang et al. (2026) for the simple tasks and the input blocks of Sec. 5.3 for the complex tasks, for items that the model answers correctly when the task is given. Their unpatched accuracy is the baseline.

2. *Mean output*: for each head in the set, the mean of $c_{l,h}$ over the first 100 prompts of a wording answered correctly, leaving out the patched items. As in the code of Yang et al.

(2026), each wording of a style (each instruction template, each number of shots, each rule wording) gives its own mean and its own patch.

3. *Patch*: at the last position of each patched prompt, the output of every head in the set is replaced by its mean output times a scaling factor α: for the simple tasks $\alpha \in \{-10, -6, -4, -2, 0, 1, 2, \ldots, 10\}$, the values of Yang et al. (2026); for the complex tasks, whose replies run to 1,024 tokens, their even values and 1, $\alpha \in \{-10, -6, -4, -2, 0, 1, 2, 4, 6, 8, 10\}$. The model answers from the patched prompt, generated as the saved outputs of Sec. 5.5, and the answer is scored by the flexible score. For the simple tasks this replaces the first generated token, which the code of Yang et al. (2026) reads; the first-token score is kept beside it.

4. *Control*: the same patch at the same heads, with the mean output of a different task. A head set carries the task if its own mean raises accuracy and the control mean does not; a gain under both points to the answer format rather than the task.

Three head sets are patched for each task and style: $H^{\mathrm{L}}_{t,s}$, $H^{\mathrm{J}}_{t,s}$, and the heads found only by J-head, $H^{\mathrm{J}}_{t,s} \setminus H^{\mathrm{L}}_{t,s}$. We report the accuracy at each α averaged over the wordings of the mean's style. For every task the patch is applied at the last prompt position only, as in Yang et al. (2026); the reply is then generated without it, and its answer is scored. Which items are patched and which means and control tasks are used are given for each task family in E1 of the companion document and in E2 (Sec. 6.2).

5.5 Answer evaluation

We score every reply in more than one way because of what Hua et al. (2025) show about scoring and prompt wording. They evaluate seven LLMs, Llama-3.1-8B-Instruct among them, on six benchmarks under twelve prompt templates each. Each reply is scored twice: once with the benchmark's usual rule (log-likelihood over the answer options, or extraction and matching of the answer string), and once by an LLM judge that sees the question, the reference answer and the reply. Under the rules, accuracy varies much more across templates than under the judge (Gemma-2 on ARC-Challenge: standard deviation 0.28 against 0.005). The ranking of the models also changes from template to template (mean Spearman correlation between templates 0.30 against 0.92). The rules mark answers wrong that are correct but phrased or formatted differently. Much of the measured sensitivity to prompt wording therefore comes from the scoring, not from the model. A rule built to accept equivalent answers (symbolic equivalence on MATH) varies as little as the judge. The judge's verdicts agree with those of three human annotators (Cohen's κ 0.92 over all benchmarks, lowest 0.67 on NarrativeQA) and with those of a second judge model. This matters here because E0 compares prompt styles and wordings by accuracy, and a difference between wordings may only be a difference in how the answers are written. Both source evaluations we use are rigid matchers. For the simple tasks, Yang et al. (2026) compare only the first generated token with the first token of the single reference answer. A capitalized German noun (*Gemeinde* for the reference *gemeinde*), an answer in quotes, or another valid synonym is therefore wrong, and a first token that only begins the reference (*Niger* for *Nigeria*) is right. For a numeric reference the first token is the leading space alone (the tokenizer splits `" 2"` into `" "` and `"2"`), so any number counts as correct: in E0 all 29 numeric items of next-item and of prev-item are scored correct under every instruction template. For the complex tasks, the parser of Fu et al. (2026) reads only the first `\boxed{}`, so a correct answer written outside the box is wrong, and the short output limits of their base-model runs (2 to 50 tokens) cut longer answers off. In a first check on the stored E0 replies, the accuracy of Operator Function across its rule wordings (0.01 to 0.99) becomes 1.00 under every wording when an unboxed reply is scored by its last integer, and on the translation and next-item tasks most of the spread across instruction templates goes once case, quotes and leading articles are

ignored. We therefore evaluate every E0 prompt in two ways, from one saved output, and also with a flexible score that ignores how the answer is written.

1. *Saved output.* Every reply is generated once, with greedy decoding, and saved with its prompt, input, reference answer, generated token ids and text, number of tokens, and why generation stopped, so that every score and analysis below can be recomputed without running the model again. Each group of tasks has one generation scheme.

   - *Simple tasks*: raw text, no chat template, in batches of 20 as in Yang et al. (2026), with the padding masked. A reply ends at the first of: the delimiter `;` of the example prompts, a newline once the reply has begun, the end-of-sequence token, or 32 new tokens, twice the longest reference answer (15 tokens).
   - *Complex tasks*: under the chat template, the same for all five tasks. A reply ends at the end-of-sequence token or at 1,024 new tokens (Sec. 5.3).

   Under greedy decoding the first tokens of a reply do not depend on how long generation runs, so the reply cut to a shorter limit is the reply that limit would give. Cut to the limits of Fu et al. (2026, App. A), it also shows which answers are given straight away.

2. *Strict score*: the source evaluation, unchanged, on the full reply: for the simple tasks, the first generated token against the first token of the reference (Yang et al., 2026); for the complex tasks, the scoring of Sec. 5.3. Computed on the replies cut to the E0 limits, it must reproduce the E0 accuracies; differences can only come from batch numerics and are reported.

3. *Flexible score* (simple tasks): the reply, up to where it stops, begins with the whole reference answer, as words. Case, accents, punctuation and one leading article or *to* are ignored. A hyphen or apostrophe inside a word joins it (*non-corresponding* matches *non-corresponding*), a possessive *'s* is dropped, and any other punctuation separates words (*petit(e)* matches *petit*); a part of the reference in parentheses is optional (*Euro* matches *Euro (EUR)*). It removes the formatting errors of the first-token score (case, quotes, articles) and its false positives (*Niger* for *Nigeria*). Another spelling of a name (*Luxembourg* for *Luxembourg City*) and another valid answer remain wrong; these are left to the judge.

4. *Flexible score* (complex tasks): the answer the strict score looks for, in any notation. It is rewritten in the format of Fu et al. (2026) and scored by the strict score, so the checks of the answer (for Set Game, that the cards are on the board and form a set) stay theirs. Operator Function: the number in the first `\boxed{}`, ignoring other text in the box, or, if the reply has no box, the number on its last line. Set Game: the first box that holds three readable cards; the brackets around a card may be missing or unmatched. Noun Class Agreement and Lexical Category Inference: the reader Fu et al. (2026) use for Noun Class Agreement (a line that is only Yes or No, else a line beginning "Answer: Yes"), without its last resort, the first yes or no anywhere in the reply, which in replies without task information reads words such as "(no category)"; a closing "the answer is Yes" is also read. Tapatan: as the strict score; its replies write the labels exactly, and paraphrases ("A wins") also occur inside the reasoning, before the final label. Where a reply has more than one box or label, the first is read, as by the strict score: a later one is often a second attempt or, under example prompts, the label of a new board the model makes up.

5. *Judged score*: an LLM judge (GPT-5.5) receives a neutral description of the task (not the prompt wording being evaluated), the item's input, the reference answer and the full reply, but not the strict score. It returns whether the reply's final answer is correct, the answer it extracted, and, where it disagrees with the strict score, the reasons (below). The judge evaluates the final answer, not the reasoning, accepts a different answer when the task

admits one (another synonym, antonym or translation, another valid set on the board), and counts two conflicting final answers as wrong. Every reply is judged, including those the strict score accepts. Where a rule can check the extracted answer, the judge's verdict is compared with it: integer comparison for Operator Function, the set check of Fu et al. (2026) on the board for Set Game, and the label for Tapatan, Noun Class Agreement and Lexical Category Inference.

6. *Straight away*: for the simple tasks this is now measured instead of assumed: the reply begins with the answer the judge extracted, after leading whitespace, rather than with a quote, an article or another preamble.

Reasons for disagreement. Each reply that the strict score marks wrong and the judge right gets one or more reasons from three groups; its primary reason is the first that applies, in the order below. Reasons that a rule can detect (a missing box, case, punctuation) are assigned by rule; the judge assigns the others.

1. *Where the answer is*: the strict score reads a part of the reply that does not hold the final answer.
    - *No box*: the answer is not in a `\boxed{}` (Set Game, Operator Function).
    - *Leading text*: text before the answer (*The answer is*, an article, the input repeated) where the score reads the first token or word (simple tasks, Lexical Category Inference).
    - *Earlier answer*: the score read an earlier answer, which the model then revised or restated correctly.
2. *How the answer is written*: the score finds the answer but does not accept its form.
    - *Case*: the answer differs from the reference only in capitalization.
    - *Extra characters*: quotes, a final period, Markdown or LaTeX around the answer, or extra text inside the box (`\boxed{x = 268}`).
    - *Tokenization*: the same word, whose first token differs from the first token of the reference.
    - *Surface variant*: the same answer written differently: diacritics, number format or spelled-out numbers, inflection, abbreviation, another card notation, a paraphrased label (*Player A wins*).
3. *Which answer is right*: a *valid alternative*, a different answer that is also correct, which a single reference cannot accept; or a *reference error*, a reference that appears wrong, checked by hand.
4. *Other*.

Replies that the strict score marks right and the judge wrong get reasons from the same groups: a *first-token false positive* (a first token that only begins the reference, as *Niger* for *Nigeria*) is a matter of how the answer is written, and *parser luck* (a label mentioned in passing before a different final answer) of where it is. Answers beyond the source limit are no disagreement, since both scores read the full reply; they are counted by comparing the strict score on the reply cut to the source limit with that on the full reply.

What changes. Head identification, $P^{+}_{t,s}$ and the causal checks use the flexible score, so the selection of the identification wordings, the head sets, the means, the patched items and the scoring of the patched replies are redone together. The judged score is not used for them: it also accepts other valid answers, which the reference of the patched items does not contain. Under the flexible score the most accurate instruction template changes on antonym (template 3 to 2), country–capital (1 to 3) and English–German (3 to 1), and the most accurate rule wording on Lexical Category Inference (wording 2 to 6) and Operator Function (wording 5 to the original prompt: all seven score 1.00, and the first is taken).

Second check. The code of Yang et al. (2026) releases one script for this check (`causal_fix_incorrect.py`), and it differs from the procedure described in their App. G.1, which we follow above, in the patched prompts. Instead of prompts without task information, it patches prompts that give the task and are answered wrongly: the 1-shot prompts of the items answered wrongly with one shot. These prompts show the answer format, and their unpatched accuracy is 0 by construction. Head sets, means, α and scoring are as above, and the script has no control. We run it as a second check and report both. On a task with two labels, a patch that only moves the model's answers towards one label already raises accuracy on these items, to the share of that label among them; we compare the gains with this share.

A control for the second check. For the complex tasks we add the control mean of the first check to the second. A prompt answered wrongly once can be answered correctly when anything in its forward pass changes. On Set Game and Tapatan the model reasons for hundreds of tokens before it answers, and a change at the last prompt token sends that reasoning down another path, which sometimes ends in the right answer. On the two-label tasks, any patch that moves the answers towards one label turns the items whose reference is that label into correct ones. Neither gain needs the task. In a first run of the second check without a control, setting the heads to zero ($\alpha = 0$) fixed 21 to 23 of the 100 wrongly answered Set Game items and 13 to 17 of Tapatan's, and on Noun Class Agreement and Lexical Category Inference the largest gains came where a patch gave one label to every item. A head set therefore counts as carrying the task in the second check only where its own mean raises accuracy more than the control mean does at the same heads, comparing the highest accuracy of each over α, as in the first check. The simple tasks keep the check without a control, as in the released script: their open answers cannot be reached by a shift towards one label, and their replies are a few tokens long.

5.6 Measures

For each task t, style s and readout R:

- *Accuracy* of each prompt, with the evaluation of the source paper: that of Yang et al. (2026) for the simple tasks and that of Fu et al. (2026) for the complex tasks. In E0 also the judged accuracy and the flexible accuracy (Sec. 5.5).
- *Head count* $|H^{R}_{t,s}|$ and the distribution of the heads over layers.
- *Overlap* between two head sets A and B: $|A \cap B|/|A|$ and $|A \cap B|/|B|$. Compared pairs: $H^{\mathrm{L}}_{t,s}$ and $H^{\mathrm{J}}_{t,s}$ (RQ2); $H^{R}_{t,\mathrm{rule}}$ and $H^{R}_{t,\mathrm{example}}$ (RQ3).
- *Causal effect*: the highest accuracy over α minus the unpatched accuracy, for the head set and for the control.
- *Copy share*: the share of prompts on which a head's matched terms ($T^{R}_{k,l,h}(p) \cap V_t$) occur in the prompt text, averaged over the heads of $H^{R}_{t,s}$. It separates heads that may copy task words from the prompt from heads that do not.

6 Experiments

Every experiment is run once, with greedy decoding, on Llama-3.1-8B-Instruct. Each one is described below by the conditions it crosses and the steps it runs; the methods of Sec. 5 are referred to where they are defined.

6.1 E0: Behavioural baseline

Question. On which tasks and prompt styles does the model perform above chance, and does it answer straight away?

Conditions. The five tasks at the easy level, each on the same 300 items (600 for Lexical Category Inference), under ten prompts: the original rules prompt and its six wordings (rule style), the example-based prompt, the combined prompt, and the input block alone (no task information, Sec. 5.3).

Procedure.

1. Generate the model's answer to every prompt and score it with the evaluation of the source paper, as in Sec. 5.3.
2. Record whether the answer is the first generated text (answered straight away).
3. Compare each accuracy with chance: the share of the most frequent label for the tasks with a fixed label set, 0 for open answers.
4. Evaluate every prompt again from its saved output (Sec. 5.5): the strict score, the flexible score, the judged score, and the reasons for each disagreement.

Output. Accuracy per task and prompt (Table 1); for every task and prompt, the strict and judged accuracy, the share answered straight away, the four counts of agreement and disagreement between the two scores, and the disagreements by reason; for every task, the standard deviation of accuracy across the prompts of a style under each score (Hua et al., 2025), the Spearman correlation between the two scores' rankings of these prompts, and whether the most accurate rule wording changes; for each task and style, the prompts $P^{+}_{t,s}$ used by E2 and, for the rule style, the most accurate wording from which they are taken; the items answered correctly and wrongly, which select the patched items of the causal checks.

6.2 E2: Complex tasks

Question. RQ1 and H1, RQ2 and H2, and RQ3: are lexical task heads found and causally effective for complex tasks, under which readout, and are they shared between rule-based and example-based prompts?

Conditions. 5 tasks $\times$ 3 styles (rule, example, combined) $\times$ 2 readouts, with the same three head sets per pair of task and style as in E1 of the companion document. All head sets are read at the last prompt token.

Procedure.

1. *Identification prompts*: for each style, the first 100 prompts in $P^{+}_{t,s}$ (Sec. 4): under the most accurate of the seven rule wordings, under the example-based prompt and under the combined prompt. Where fewer prompts qualify, all of them are used.

2. *Head sets*: Definition 1 with the term list of the task (Sec. 5.3).

3. *Measures*: as in E1 of the companion document, with the overlap of the rule-style and example-style sets for RQ3.

4. *First causal check*, rule-style head sets only. Patched items: the first 100 items in P^+ under the original rules prompt. Patched prompts: their prompts without task information (the input blocks of Sec. 5.3). Means: one per rule wording, seven in all, each over the first 100 prompts of that wording in P^+, leaving out the patched items. Control: the mean of a task with the same answer format under its original rules prompt (Noun Class Agreement and Lexical Category Inference with each other, Set Game and Operator Function with each other, Tapatan with Set Game). Scaling: the 11 values of α of Sec. 5.4.

5. *Second causal check*, rule-style head sets only. Patched items: the first 100 items answered wrongly under the original rules prompt. Patched prompts: their original rules prompts, whose unpatched accuracy is 0 by construction. Operator Function has no second check: under the flexible score every item is answered correctly under its original rules prompt. Means, control mean and α as in the first check (Sec. 5.4, "A control for the second check"). On the two-label tasks, the accuracy of answering every patched item with one label, the share of that label among the patched items, is the reference that a shift towards one answer reaches without any task information.

6. *Scoring*: the reply generated from each patched prompt, as the saved outputs of the complex tasks, scored by the flexible score. Accuracy at each α is averaged over the seven means; the causal effect is computed as in E1 of the companion document.

Output. Head sets and measures (Table 3, Figures 2 and 3); causal effects (Tables 4 and 5); accuracy against α (Figure 1).

7 Results

Tables 1–5 and Figures 2–1 are generated from the result files by `scripts/results_tex.py`. E0 accuracies are the strict scores of the saved outputs (Sec. 5.5), under the generation and scoring of Sec. 5.3. Head sets and causal checks use the flexible score (Sec. 5.5, "What changes"): P^+, the means and the patched items are chosen by it, and causal-check accuracies are the flexible scores of the replies generated from the patched prompts, over the patched items.

7.1 E0: Behavioural baseline

Complex tasks (Table 1). Noun Class Agreement and Lexical Category Inference are answered straight away and above chance under all seven rule wordings and under examples. On combined prompts the model often explains before answering: it answers straight away on 64% (Noun Class Agreement) and 16% (Lexical Category Inference) of them, against at least 97% under every rule and example prompt. The parser of Noun Class Agreement still finds the answer after an explanation, so its combined accuracy is close to that of its rule prompts; that of Lexical Category Inference reads only the first word, so its combined accuracy falls far below chance. Without task information the model never answers straight away, and almost no reply is scored correct.

Set Game, Operator Function and Tapatan are not answered straight away under any prompt (one Tapatan reply of 2,100 under the rule wordings): the model reasons first, and Table 1 scores the answer it reaches. Set Game and Operator Function are solved under rules and combined prompts, and from examples alone on one item of 300 each; with examples the model often analyses the demonstrations until the 1,024-token limit (147 Set Game and 58 Operator

		Rule					No task	
Task	Items	orig.	w1–6	range	Examples	Combined	info	Chance
Noun Class Agreement	300	0.61	0.58	0.55–0.61	0.58	0.55	0.01	0.50
Lexical Category Inference	600	0.67	0.66	0.64–0.68	0.59	0.11	0.00	0.50
Set Game	300	0.58	0.57	0.51–0.64	0.00	0.54	0.32	0.00
Operator Function	300	0.98	0.56	0.01–0.99	0.00	0.99	0.00	0.00
Tapatan	300	0.43	0.39	0.36–0.43	0.29	0.16	0.00	0.34

Table 1: E0, complex tasks at the easy level: strict accuracy per prompt, on the whole reply up to 1,024 tokens (Sec. 5.3). Rule: the original rules prompt (orig.) and the six added wordings (w1–6: mean, and range). No task info: the prompt without task information. Chance: share of the most frequent label for the tasks with a fixed label set, 0 for open answers.

		Rule					No task	
Task	Items	orig.	w1–6	range	Examples	Combined	info	Chance
Noun Class Agreement	300	0.61	0.58	0.55–0.61	0.58	0.55	0.00	0.50
Lexical Category Inference	600	0.67	0.67	0.64–0.69	0.59	0.76	0.00	0.50
Set Game	300	0.61	0.60	0.52–0.68	0.00	0.54	0.49	0.00
Operator Function	300	1.00	1.00	1.00–1.00	0.11	1.00	0.00	0.00
Tapatan	300	0.43	0.39	0.36–0.43	0.29	0.16	0.00	0.34

Table 2: E0, complex tasks: flexible accuracy per prompt (Sec. 5.5), on the same saved replies and with the same layout as Table 1.

Function replies). Operator Function varies most across wordings: every wrong answer under its rule wordings is a reply without a `\boxed{}` answer, which the scoring counts as wrong, and under wording 2 only 3 of 300 replies have one. Tapatan stays close to chance under all seven rule wordings and falls below it with examples and combined prompts, so its P^+ sets likely include items answered correctly by chance. Set Game is solved on 97 of 300 items without task information: its input block still asks which three cards on the board form a set.

Under the flexible score (Table 2), Operator Function is solved on every item under all seven rule wordings and the combined prompt, so its spread across wordings came entirely from the box; from examples alone it is now solved on 34 of 300 items, answered without the box the examples show. Set Game gains 1 to 6 points per rule wording, all from boxes with a missing or unmatched bracket, and 51 items without task information, where the cards are often written without brackets. The combined prompts of Lexical Category Inference rise from 0.11 to 0.76, above every rule wording: the model explains first and gives its answer at the end, which the first-word scoring misses; two rule wordings gain 10 of 600 items in the same way. Noun Class Agreement is unchanged, except that its prompts without task information lose their 3 correct replies, read by the last resort the flexible reader drops. Tapatan is unchanged by construction. The most accurate rule wording, from which P^+ is taken, changes on two tasks under this score: on Operator Function all wordings tie at 1.00, and on Lexical Category Inference wording 6 passes wording 2 by 2 of 600 items. Head identification and the causal checks use the flexible score.

7.2 E2: Complex tasks

Head sets (Table 3). Both readouts find lexical task heads for every complex task and style. The sets are larger than for the simple tasks, against H1, but the head counts of the two families are not comparable. The complex term lists expand to 114 to 340 token ids and the simple ones to 2 to 22, and a longer list makes a match among a head's ten top tokens more likely for any head. Small P^+ sets add to this: the example-style P^+ of Set Game is one prompt, so every

head that matches on it is a lexical task head (9 and 13 heads), and for the example style of Operator Function (34 prompts) and of Tapatan (88) and Tapatan's combined style (47), 4 to 9 matching prompts suffice. As on the simple tasks, J-head finds more heads than Logit-head on every pair, and three fifths of its added heads lie in layers 10 to 19, while four fifths of the Logit-head heads lie in layer 20 or later. Its set covers 44% to 91% of the Logit-head set, never all of it. Of each example-style set, 48% to 94% is also in the rule-style set, at least half on 9 of the 10 pairs of task and readout (RQ3); the exception is Operator Function under J-head, and the example-style sets of Set Game rest on one prompt. Copy shares lie between about half and nine tenths under rule and combined prompts and are lower under example prompts: zero on Set Game and Operator Function, and above the near-zero values of the simple tasks' example prompts on the other three tasks (Figure 3).

Causal check (Tables 4 and 5, Figure 1). In the first check, the prompts without task information get no answer on Noun Class Agreement, Lexical Category Inference, Operator Function and Tapatan: the model describes the input (a sentence in a coded language, items to classify, a board it takes for Sudoku or tic-tac-toe) or, on Operator Function, asks for the function. No patch changes this. Averaged over the seven means, accuracy stays at zero on the first three tasks, where some Operator Function replies box a number that is not the answer, and reaches at most two items on Tapatan, with task and control means alike; under any single mean and α it never exceeds three items. On Set Game, 58 of the 100 patched items are solved without task information, since the input block still asks which cards on the board form a set. Averaged over the seven means, the patches raise this by at most five items, and the control mean by as much or more. The first check shows no causal effect of the rule-style head sets on any complex task.

In the second check, every head set raises accuracy on the four tasks with wrongly answered items; Operator Function has none, since under the flexible score it answers every item correctly under its original rules prompt. The control mean (Sec. 5.4, "A control for the second check") raises accuracy as much. Its highest accuracy at the same heads is within two items of the task mean's on Set Game, as high or higher on Lexical Category Inference and Tapatan, and higher on Noun Class Agreement except at the Logit-head set. At $\alpha = 0$ both means set the heads to zero and give the same accuracy, which already fixes 21 to 23 of the 100 Set Game items and 13 to 17 of Tapatan's. On the two-label tasks, the largest gains under either mean come where a patch moves the answers towards one label or to the other one. With the task means, the Logit-head set of Noun Class Agreement answers "Yes" to every item at $\alpha = 10$ (0.58, the share of "Yes" among them), the only gain above the control there; on Lexical Category Inference every set answers "No" to all items at $\alpha = 4$ (0.28) and "Yes" to most at $\alpha = 8$ (0.65 to 0.71, against 0.72 for answering "Yes" to all). The control mean at the J-head set of Noun Class Agreement reaches 0.80 at $\alpha = -10$, above that line, because it turns 80 of the 100 answers into the other label, which on these wrongly answered items is the right one. On Set Game, at α of 8 and 10 the Logit-head and J-head sets solve 47 to 55 items with the task means and 32 to 42 with the control mean, but the control mean reaches the task means' level at α of -6 or -10. With the task means at $\alpha = 10$, the model writes a set almost at once (Logit-head set: median reply 97 characters, against 868 at $\alpha = 1$). The second check shows no gain of a task mean that the control mean does not match.

Hypotheses. For RQ1, the method finds lexical task heads for every complex task under both readouts, but neither check shows a task-specific effect for them: the first check changes nothing on four tasks and nothing beyond the control on Set Game, and the gains of the second check are matched by the control mean, which needs no task information either. This agrees with the causal part of H1; its head-count part cannot be tested with term lists of such different lengths. H2 fails on both conditions: the J-head set misses part of the Logit-head set on every pair, and

	Rule				Example				Combined				Rule ∩ ex.	
Task	$\|P^+\|$	L	J	L∩J	$\|P^+\|$	L	J	L∩J	$\|P^+\|$	L	J	L∩J	L	J
Noun Class Agreement	100	15	26	10	100	12	17	6	100	21	36	18	7	12
Lexical Category Inference	100	43	53	35	100	16	22	10	100	49	65	42	15	13
Set Game	100	55	75	48	1	9	13	4	100	52	65	40	6	8
Operator Function	100	23	40	21	34	14	21	10	100	22	39	20	7	10
Tapatan	100	61	77	46	88	46	62	38	47	55	70	48	32	40

Table 3: E2: lexical task heads per task and style. $|P^+|$: identification prompts, the first (at most 100) answered correctly under the most accurate rule wording (rule; wording 6 for Noun Class Agreement and Lexical Category Inference, wording 5 for Set Game, the original prompt for Operator Function, whose seven wordings tie, wording 1 for Tapatan), under the example-based and under the combined prompts. L: Logit-head set; J: J-head set; L∩J: heads in both sets. Rule ∩ ex.: heads in both the rule-style and the example-style set of a readout.

			Task mean			Control mean		
Task	Items	Baseline	L	J	J only	L	J	J only
Noun Class Agreement	100	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Lexical Category Inference	100	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Set Game	100	0.58	0.04	0.04	0.05	0.06	0.05	0.10
Operator Function	100	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Tapatan	100	0.00	0.01	0.01	0.01	0.01	0.00	0.02

Table 4: E2 causal check: causal effect, the highest accuracy over α minus the unpatched accuracy (baseline); accuracy is the flexible score of the reply generated from each patched prompt. Task mean: the head set's mean output over the task's own prompts; L: Logit-head set; J: J-head set; J only: heads in the J-head set and not in the Logit-head set. The rule-style head sets, patched with the means of the seven rule wordings into the prompts without task information; accuracy is averaged over the seven means before the maximum is taken. Control mean: the control task's mean under its original rules prompt. 100 patched items per task, the 11 values of α of Sec. 5.4.

the heads found only by J-head have no task-specific effect.

8 Plan

Steps.

1. Build the additions of Sec. 5.3: rule wordings (each checked to state the same rules as the original prompt), term lists, prompts without task information.

2. Run E0 and select tasks. Evaluate its saved outputs with the strict and the judged score (Sec. 5.5).

3. Run E2.

Reused code and data: the lexical-task-head code of Yang et al. (2026); the Head-JLens code and matrices of Merlo (2026); the data and generation code of Fu et al. (2026); the task datasets of Todd et al. (2024).

Limitations. One model, and one difficulty level of each complex task. The term lists are generated by an LLM and checked by hand, and the identified heads depend on them. Rule-based prompts contain task words, so a head may decode them by copying from the prompt; the

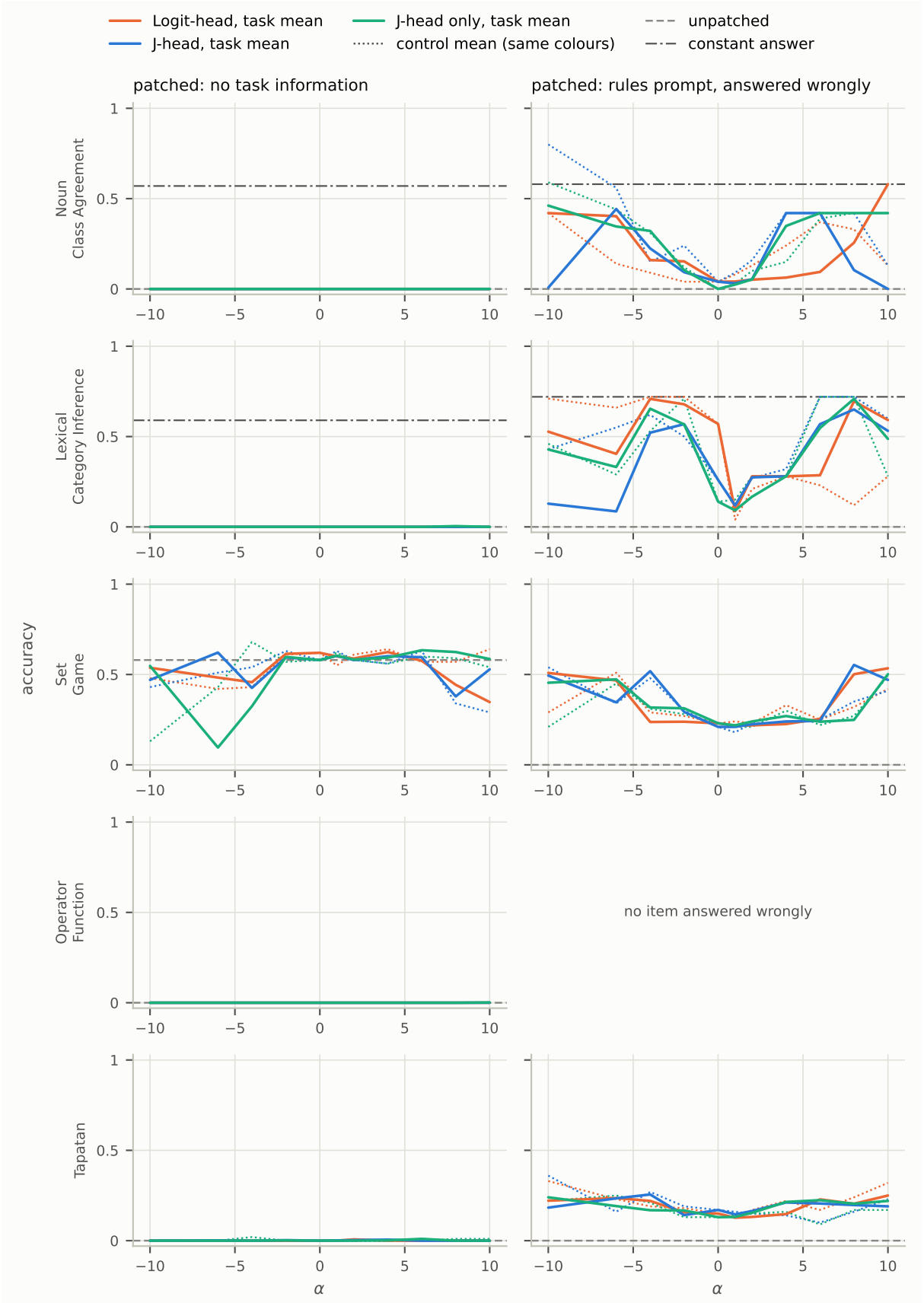


Figure 1: E2 causal checks: accuracy on the patched items against α. Solid: the head set's mean over the task's prompts; dotted: the control mean at the same heads; dashed: unpatched accuracy. The rule-style head sets, with accuracy averaged over the means of the seven rule wordings. Left: the prompts without task information patched (first check); right: the original rules prompts answered wrongly (second check), 100 patched items per task; Operator Function has no second check. Constant answer: the accuracy of answering every patched item with its more frequent label.

			Task mean			Control mean		
Task	Items	Baseline	L	J	J only	L	J	J only
Noun Class Agreement	100	0.00	0.58	0.44	0.46	0.42	0.80	0.59
Lexical Category Inference	100	0.00	0.71	0.65	0.71	0.72	0.72	0.72
Set Game	100	0.00	0.53	0.55	0.50	0.51	0.54	0.49
Operator Function	*no item answered wrongly*							
Tapatan	100	0.00	0.25	0.26	0.24	0.33	0.36	0.25

Table 5: E2, second causal check, as Table 4, with the original rules prompts of the first 100 items answered wrongly under them patched; Operator Function answers every item correctly under its original rules prompt and has none. Unpatched accuracy is 0 by construction. Control mean as in Table 4 (Sec. 5.4, "A control for the second check"). Answering every patched item with its more frequent label scores 0.58 on Noun Class Agreement and 0.72 on Lexical Category Inference ("Yes" for both).

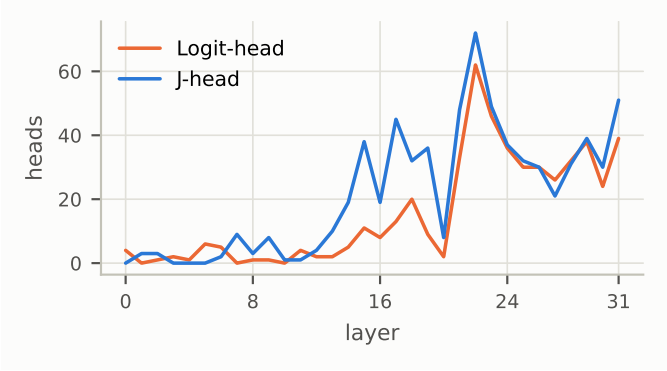


Figure 2: Lexical task heads per layer under each readout, summed over the head sets of all complex tasks and styles, at the last prompt token.

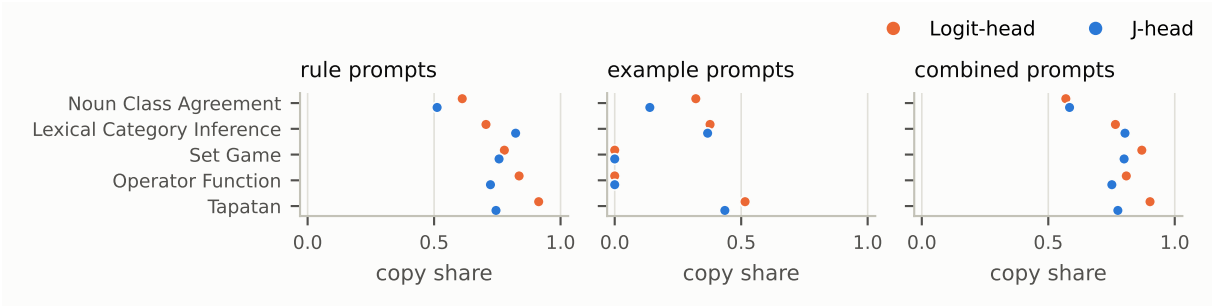


Figure 3: Copy share of each complex-task head set (Sec. 5.6): the share of prompts on which a head's matched terms occur in the prompt text, averaged over the set, at the last prompt token; missing points are empty sets.

copy share measures this but does not decide it. Under J-head, some sites rank tokens highly on unrelated prompts as well (Merlo, 2026, Sec. 3.6), so a larger J-head set is not by itself evidence of more task representation; the causal check is needed. P^+ and the patched items are chosen by the flexible score, which accepts only the reference answer: a prompt answered with another valid answer is counted as wrong. The judged score depends on one LLM judge, which is not checked against hand labels here; Hua et al. (2025) find that LLM judges agree with human annotators on other tasks.

Deliverables. Head sets, measures and causal-check curves for every task, style and readout.

References

Guy Davidson, Todd M. Gureckis, Brenden M. Lake, and Adina Williams. Do different prompting methods yield a common task representation in language models? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.12075.

Xiang Fu, Seungmin Cho, Yukyung Lee, and Najoung Kim. LLMs learn better in-context from rules than from examples. *arXiv preprint arXiv:2609.03213*, 2026.

Wes Gurnee, Nicholas Sofroniew, Adam Pearce, et al. Verbalizable representations form a global workspace in language models. Transformer Circuits Thread, 2026. URL https://transformer-circuits.pub/2026/workspace/index.html.

Andong Hua, Kenan Tang, Chenhe Gu, Jindong Gu, Eric Wong, and Yao Qin. Flaw or artifact? rethinking prompt sensitivity in evaluating LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. arXiv:2509.01790.

Filippo Merlo. Reading attention heads through the Jacobian lens. Manuscript, October 2026. https://github.com/filippo-merlo/head_level_jlens, 2026.

nostalgebraist. interpreting GPT: the logit lens. LessWrong, 2020. URL https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens.

Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.15213.

Zhuonan Yang, Jacob Xiaochen Li, Francisco Piedrahita Velez, Eric Todd, David Bau, Michael L. Littman, Stephen H. Bach, and Ellie Pavlick. Shared lexical task representations explain behavioral variability in LLMs. In *International Conference on Machine Learning (ICML)*, volume 306 of *PMLR*, 2026. arXiv:2604.22027.

A Original rule prompts

The rules-only prompts of Fu et al. (2026), from their released code (https://github.com/tinlaboratory/rules-vs-examples, `rules-mode-prompts/`, MIT licence). Text in curly braces is filled per item. Tapatan uses the final-board-state prompt, whose levels differ only in its parameters: board size 3, 5, 7 and line length 3, 4, 5 for Easy, Medium, Hard (Fu et al., 2026, Table 1); the version in Fu et al. (2026, App. F) also states the maximum number of pieces per player and that at most one player has a winning line. The Medium and Hard prompts of Lexical Category Inference are identical within each logic variant.

Set Game

Easy.

```
You will be shown 9 cards on the board and have to select 3 cards that form a GAME-SET based
↪  on the following rules.

RULES:
1. Each card has two attributes: an animal and a biome.
2. There are 10 types of animals: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. There are 10 types of biomes: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. A GAME-SET is a set of three cards: For each attribute (animal, biome), the three cards
↪  must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have the same
↪  value, and 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Medium.

```
You will be shown 9 cards on the board and have to select 3 cards that form a GAME-SET based
↪  on the following rules.

RULES:
1. Each card has three attributes: an animal, a biome, and a food.
2. There are 10 types of animals: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. There are 10 types of biomes: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. There are 10 types of food: apple, potato, bread, cake, carrot, mutton, beef, cookie,
↪  pie, melon.
5. A GAME-SET is a set of three cards: For each attribute (animal, biome, food), the three
↪  cards must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have the same
↪  value, and 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Hard.

```
You will be shown 9 cards on the board and have to select 3 cards that form a GAME-SET based
↪  on the following rules.

RULES:
1. Each card has a number and three attributes: an animal, a biome, and a food.
```

```
2. Numbers range from 1 to 12.
3. There are 10 types of animals: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
4. There are 10 types of biomes: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
5. There are 10 types of food: apple, potato, bread, cake, carrot, mutton, beef, cookie,
↪  pie, melon.
6. A GAME-SET is a set of three cards: For each attribute (animal, biome, food), the three
↪  cards must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have the same
↪  value, and 1 of them has a different value, the set is NOT valid.
7. But only for the number, 2 of the cards should have the same number, and 1 of them should
↪  have a different number in order for the set to be valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Tapatan

all levels.

```
You will be learning a new board game. You will then be given one final board state from
↪  that game. Determine the current game outcome.

RULES:
1. The board is a {board_size}x{board_size} grid of points with coordinates (x,y), where 0
↪  <= x,y < {board_size}.
2. A board cell contains A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins if they have {line_length} pieces in a straight line (horizontal, vertical,
↪  or diagonal) and each consecutive pair of points in that line is adjacent under rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Operator Function

Easy.

```
You will be shown a problem that uses a newly defined operator function based on the
↪  following rules and need to calculate the answer for the problem.

RULES:
1. The new operator function f is composed of two arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z be inputs of the function.
3. To compute f(x,y,z), first multiply x and y.
4. Then, add z to the result from the previous operation.
```

```
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Medium.

```
You will be shown a problem that uses a newly defined operator function based on the
↪  following rules and need to calculate the answer for the problem.

RULES:
1. The new operator function g is composed of three arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z, a be inputs of the function.
3. To compute g(x,y,z,a), first subtract y from x.
4. Second, multiply the result from the previous operation by z.
5. Last, adds a to the result of the previous operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Hard.

```
You will be shown a problem that uses a newly defined operator function based on the
↪  following rules and need to calculate the answer for the problem.

RULES:
1. The new operator function h is composed of four arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z, a, b be inputs of the function.
3. To compute h(x,y,z,a,b), first add x and y.
4. Second, multiply the result from the previous operation by z.
5. Third, subtract a from the result of the previous operation.
6. Last, take the remainder when the result is divided by b.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Noun Class Agreement

Easy.

```
You will be given one sentence in a synthetic language. Decide whether the sentence is
↪  grammatical.

RULES:
1. There are 2 noun classes: A and B.
2. Each noun phrase has the form DET NOUN.
3. The class A determiner is ka, and the class B determiner is ti.
4. The class A nouns are tac, sular, fep, wug. The class B nouns are bim, noko, glarn, zesh.
5. Sentences follow the template DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical if and only if, in each noun phrase, the determiner matches the
↪  noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Medium.

```
You will be given one sentence in a synthetic language. Decide whether the sentence is
↪  grammatical.

RULES:
1. There are 4 noun classes: A, B, C, D.
2. A noun phrase has the form DET ADJ-SUFF NOUN.
3. The class A determiner is ka, the class B determiner is ti, the class C determiner is su,
↪  and the class D determiner is vo.
4. Adjectives have the form STEM-SUFFIX. The suffix must match the noun's class.
5. The class A adjective suffix is -en, the class B adjective suffix is -os, the class C
↪  adjective suffix is -im, and the class D adjective suffix is -at.
6. Example adjective tokens are glim-en, prun-os, fen-im, zay-at.
7. The class A nouns are tav, fap. The class B nouns are bim, glarn. The class C nouns are
↪  sular, wug. The class D nouns are noko, zesh.
8. Sentences follow the template DET ADJ-SUFF NOUN VERB DET ADJ-SUFF NOUN.
9. Verbs are plain roots like dax, miv, lorp and never change.
10. A sentence is grammatical if and only if, in each noun phrase, both the determiner and
↪  the adjective suffix match the noun's class.
11. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Hard.

```
You will be given one sentence in a synthetic language. Decide whether the sentence is
↪  grammatical.

RULES:
1. There are 6 noun classes: A, B, C, D, E, F.
2. A noun phrase has the form DET ADJ-SUFF NOUN.
3. The class A determiner is ka, the class B determiner is ti, the class C determiner is su,
↪  the class D determiner is vo, the class E determiner is ne, and the class F determiner
↪  is la.
4. Adjectives have the form STEM-SUFFIX. The suffix must match the noun's class.
5. The class A adjective suffix is -en, the class B adjective suffix is -os, the class C
↪  adjective suffix is -im, the class D adjective suffix is -at, the class E adjective
↪  suffix is -uk, and the class F adjective suffix is -esh.
6. The verb has the form PREFIX-ROOT-SUFFIX.
7. The prefix must match the class of the subject noun, and the suffix must match the class
↪  of the object noun.
8. The class A subject prefix is ge-, the class B subject prefix is du-, the class C subject
↪  prefix is ri-, the class D subject prefix is zo-, the class E subject prefix is pa-, and
↪  the class F subject prefix is li-.
9. The class A object suffix is -an, the class B object suffix is -eb, the class C object
↪  suffix is -ig, the class D object suffix is -ot, the class E object suffix is -ul, and
↪  the class F object suffix is -er.
10. Example verb tokens are ge-dax-eb, pa-miv-ig, du-lorp-er.
11. The class A nouns are tav, fap. The class B nouns are bim, glarn. The class C nouns are
↪  sular, wug. The class D nouns are noko, zesh. The class E nouns are tac, fep. The class
↪  F nouns are lurn, prax.
12. Sentences follow the template DET ADJ-SUFF NOUN PREFIX-ROOT-SUFFIX DET ADJ-SUFF NOUN.
13. Verb roots are plain roots like dax, miv, lorp. Only the verb's prefix and suffix change.
14. A sentence is grammatical if and only if the subject noun phrase's determiner and
↪  adjective suffix match the subject noun's class, the object noun phrase's determiner and
↪  adjective suffix match the object noun's class, the verb prefix matches the subject
↪  noun's class, and the verb suffix matches the object noun's class.
15. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
```

```
Answer:
```

Lexical Category Inference

Easy.

```
You will be shown a list of four words. Decide whether this list is correct based on the
↪  following rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category has the meaning shown in the category definitions below.
3. A correct list contains exactly four comma-separated items.
4. In a correct list, the first item is a member of Category 1, the second item is a member
↪  of Category 2, and so on.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Medium and Hard, OR.

```
You will be shown a list of four words. Decide whether this list is correct based on the
↪  following rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category has the meaning shown in the category definitions below.
3. A correct list contains exactly four comma-separated items.
4. In a correct list, the first item is a member of Category 1, the second item is a member
↪  of Category 2, and so on.
5. If a category lists multiple descriptions, an item is a member of that category if it
↪  satisfies at least one listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Medium and Hard, AND.

```
You will be shown a list of four words. Decide whether this list is correct based on the
↪  following rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category has the meaning shown in the category definitions below.
3. A correct list contains exactly four comma-separated items.
4. In a correct list, the first item is a member of Category 1, the second item is a member
↪  of Category 2, and so on.
5. If a category lists multiple descriptions, an item is a member of that category only if
↪  it satisfies all listed descriptions.
```

```
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

B Generation prompts

Both prompts are given to GPT-5.5 through the Azure OpenAI API; `{rule_prompt}` is an original prompt of App. A.

Rule wordings

```
You will be given a prompt that describes the rules of a task for a language model. Write
↪  {N} paraphrases of it.

Requirements:
1. Each paraphrase must state exactly the same rules as the original: the same constraints,
↪  conditions, quantities, names, symbols and labels. Do not add, remove, weaken or
↪  strengthen any rule, and do not add examples, hints or explanations.
2. Change the wording and the sentence structure. Reorder rules only where their order does
↪  not matter; if a rule refers to another rule by its number, keep the reference correct.
3. Copy verbatim every placeholder in curly braces (e.g. {board}), the answer-format
↪  instruction, and every output label or format string (e.g. "Yes or No", "A win, B win,
↪  or continue", \boxed{...}).
4. Keep unchanged the final lines that introduce the input and the answer (e.g. "Sentence:
↪  {sentence}" and "Answer:").
5. The paraphrases must differ from the original and from each other.

Return a JSON list of {N} strings.

Original prompt:
<<<
{rule_prompt}
>>>
```

Term lists

```
You will be given the rules of a task for a language model. List words that describe the
↪  task itself: what the task asks the model to do, its operation or relation, and its key
↪  concepts. For example, for the task "give the antonym of a word": opposite, reverse,
↪  antonym, contrary.

Requirements:
1. Single words only.
2. Do not list possible answers or output labels, words that only name the items of a
↪  particular input (e.g. specific animals, nouns or numbers), or placeholders.
3. Include singular and plural forms and capitalised forms. You may include translations
↪  into other languages.
4. Return a JSON list of strings.

Task rules:
<<<
{rule_prompt}
>>>
```

C Rule wordings

The six wordings of each original rules prompt of App. A at the easy level, the level in the study, generated and checked as in Sec. 5.3. Text in curly braces is filled per item.

Set Game, easy level

Wording 1.

```
You will see 9 cards on the board, and your task is to choose 3 cards that make a GAME-SET
↪  according to these rules.

RULES:
1. Every card has two attributes: an animal and a biome.
2. The 10 animal types are: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep, turtle,
↪  wolf.
3. The 10 biome types are: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. A GAME-SET consists of three cards: For each attribute (animal, biome), the values across
↪  the three cards must be either ALL the SAME or ALL DIFFERENT. For example, if 2 cards
↪  share the same value and 1 card has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Wording 2.

```
You are given a board with 9 cards and must pick 3 cards that constitute a GAME-SET under
↪  the rules below.

RULES:
1. Each card contains two attributes: an animal and a biome.
2. The animals come in 10 types: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. The biomes come in 10 types: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. A GAME-SET is a set of three cards: For each attribute (animal, biome), the three cards
↪  must have values that are either ALL the SAME or ALL DIFFERENT. For instance, when 2
↪  cards have the same value and 1 card has another value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Wording 3.

```
A board containing 9 cards will be shown, and you need to select 3 cards that form a
↪  GAME-SET by following the rules.

RULES:
1. Each card is defined by two attributes: an animal and a biome.
2. There are 10 animal types: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. There are 10 biome types: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. A GAME-SET is made up of three cards: For each attribute (animal, biome), the three cards
↪  must be either ALL the SAME or ALL DIFFERENT. If, for example, 2 of the cards have one
↪  value and 1 has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Wording 4.

```
You will be presented with 9 cards on the board and must identify 3 cards that create a
↪  GAME-SET according to the following rules.

RULES:
1. Each card has two attributes: an animal and a biome.
2. The 10 kinds of animals are: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. The 10 kinds of biomes are: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. A GAME-SET is a group of three cards: For each attribute (animal, biome), the three cards
↪  must be either ALL the SAME or ALL DIFFERENT. e.g. when 2 of the cards share the same
↪  value and 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Wording 5.

```
From the 9 cards displayed on the board, choose 3 cards that satisfy the definition of a
↪  GAME-SET using these rules.

RULES:
1. Each card includes two attributes: an animal and a biome.
2. There are 10 types of animals: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. There are 10 types of biomes: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
```

```
4. A GAME-SET is a collection of three cards: For each attribute (animal, biome), the three
↪  cards must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have the same
↪  value while 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Wording 6.

```
You will be shown a board of 9 cards, and you have to determine 3 cards that form a GAME-SET
↪  based on the rules below.

RULES:
1. Each card has two attributes: an animal and a biome.
2. The animal attribute has 10 possible types: armadillo, axolotl, bat, cat, fox, horse,
↪  rabbit, sheep, turtle, wolf.
3. The biome attribute has 10 possible types: plain, savanna, desert, swamp, forest, jungle,
↪  taiga, hill, badland, tundra.
4. A GAME-SET is a set containing three cards: For each attribute (animal, biome), the three
↪  cards must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have matching
↪  values and 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Operator Function, easy level

Wording 1.

```
A problem involving a newly defined operator function will be presented, and you must
↪  determine its answer using these rules.

RULES:
1. The new operator function f consists of two arithmetic operators, and all of its inputs
↪  are integers.
2. Let x, y, z be the inputs to the function.
3. When computing f(x,y,z), begin by multiplying x and y.
4. After that, add z to the outcome of the preceding operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Wording 2.

```
You need to solve a problem that uses a newly introduced operator function defined according
↪  to the rules below.
```

```
RULES:
1. The new operator function f is made from two arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z represent the inputs of the function.
3. To evaluate f(x,y,z), multiply x by y first.
4. Next, add z to the result obtained from the previous operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Wording 3.

```
A problem will be given that relies on a newly defined operator function; compute the answer
↪  by applying the following rules.

RULES:
1. The new operator function f is formed using two arithmetic operators, and the inputs to
↪  it are integers.
2. Let x, y, z be inputs of the function.
3. For f(x,y,z), first take the product of x and y.
4. Then add z to the result of that prior operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Wording 4.

```
You will receive a problem using a newly defined operator function, and you must calculate
↪  the result according to these rules.

RULES:
1. The new operator function f is built from two arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z denote the inputs of the function.
3. In computing f(x,y,z), the first step is to multiply x and y.
4. The next step is to add z to the result from the previous operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Wording 5.

```
The problem shown will involve a newly defined operator function, and the answer must be
↪  found using the rules that follow.

RULES:
1. The new operator function f is composed of two arithmetic operators, with integer inputs.
2. Let x, y, z be the function's inputs.
3. To calculate f(x,y,z), start by multiplying x and y.
4. Then add z to the value produced by the preceding operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Wording 6.

```
Use the rules below to calculate the answer to a problem involving a newly defined operator
↪  function.

RULES:
```

```
1. The new operator function f contains two arithmetic operators, and the inputs are
↪  integers.
2. Let x, y, z be inputs of the function.
3. To compute f(x,y,z), multiply x and y before anything else.
4. Then add z to the result from the previous operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Noun Class Agreement, easy level

Wording 1.

```
You will receive a single sentence in a synthetic language. Determine whether it is
↪  grammatical.

RULES:
1. The language has 2 noun classes: A and B.
2. Every noun phrase is structured as DET NOUN.
3. ka is the determiner for class A, and ti is the determiner for class B.
4. The nouns in class A are tac, sular, fep, wug. The nouns in class B are bim, noko, glarn,
↪  zesh.
5. Sentences use the pattern DET NOUN VERB DET NOUN.
6. The verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical exactly when, in every noun phrase, the determiner matches the
↪  noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Wording 2.

```
You will be provided with one sentence written in a synthetic language. Judge whether the
↪  sentence is grammatical.

RULES:
1. There are 2 noun classes: A and B.
2. A noun phrase always has the form DET NOUN.
3. The determiner for class A is ka, and the determiner for class B is ti.
4. Class A contains the nouns tac, sular, fep, wug. Class B contains the nouns bim, noko,
↪  glarn, zesh.
5. A sentence follows the template DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical if and only if the determiner in each noun phrase matches the
↪  noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Wording 3.

```
You will see one sentence in a synthetic language. Decide if that sentence is grammatical.

RULES:
1. The noun classes are 2 in number: A and B.
2. Each noun phrase is of the form DET NOUN.
3. ka is used as the class A determiner, and ti is used as the class B determiner.
4. tac, sular, fep, wug are the class A nouns. bim, noko, glarn, zesh are the class B nouns.
```

```
5. Sentences have the template DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical if and only if each noun phrase has a determiner that matches
↪  the noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Wording 4.

```
A single sentence in a synthetic language will be given to you. Decide whether it is
↪  grammatical.

RULES:
1. There are 2 noun classes, A and B.
2. Noun phrases take the form DET NOUN.
3. The class A determiner is ka; the class B determiner is ti.
4. The class A nouns are tac, sular, fep, wug. The class B nouns are bim, noko, glarn, zesh.
5. Sentences are built according to DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical if and only if, for each noun phrase, the determiner matches
↪  the noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Wording 5.

```
You will get one sentence from a synthetic language. Determine whether the sentence is
↪  grammatical.

RULES:
1. There are 2 noun classes: A and B.
2. The form of every noun phrase is DET NOUN.
3. For class A, the determiner is ka, and for class B, the determiner is ti.
4. The class A nouns are tac, sular, fep, wug. The class B nouns are bim, noko, glarn, zesh.
5. Sentences conform to the template DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical if and only if the determiner matches the noun's class in each
↪  noun phrase.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Wording 6.

```
You will be shown one sentence in a synthetic language. Decide whether that sentence is
↪  grammatical.

RULES:
1. The language includes 2 noun classes: A and B.
2. Each noun phrase follows the form DET NOUN.
3. ka is the class A determiner, while ti is the class B determiner.
4. The class A nouns are tac, sular, fep, wug. The class B nouns are bim, noko, glarn, zesh.
5. Sentences follow this template: DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
```

```
7. A sentence is grammatical if and only if, within every noun phrase, the determiner
↪  matches the noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Tapatan, easy level

Wording 1.

```
You are going to learn a new board game. After that, you will receive a single final board
↪  state from the game. Decide the present game outcome.

RULES:
1. The board is a {board_size}x{board_size} grid of points whose coordinates are (x,y), with
↪  0 <= x,y < {board_size}.
2. Each board cell holds A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins when they have {line_length} pieces in one straight line (horizontal,
↪  vertical, or diagonal), with every consecutive pair of points in that line adjacent
↪  according to rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Wording 2.

```
You will be taught a new board game, and then you will be shown one final board state from
↪  that game. Identify the game outcome at that moment.

RULES:
1. The board consists of a {board_size}x{board_size} grid of points at coordinates (x,y),
↪  where 0 <= x,y < {board_size}.
2. A board cell contains A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins if they possess {line_length} pieces in a straight line (horizontal,
↪  vertical, or diagonal) and each successive pair of points in that line is adjacent under
↪  rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Wording 3.

```
You will learn the rules for a new board game. You will then be provided with one final
↪  board state from it. Determine what the current outcome of the game is.

RULES:
1. The board is a grid of points with size {board_size}x{board_size} and coordinates (x,y),
↪  where 0 <= x,y < {board_size}.
2. In a board cell, A denotes player A's piece, B denotes player B's piece, and . denotes an
↪  empty point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins by having {line_length} pieces in a straight line (horizontal, vertical, or
↪  diagonal), provided that each consecutive pair of points in that line is adjacent under
↪  rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Wording 4.

```
You will be introduced to a new board game. Next, one final board state from the game will
↪  be given to you. Work out the current game outcome.

RULES:
1. The board is a {board_size}x{board_size} grid made of points with coordinates (x,y),
↪  where 0 <= x,y < {board_size}.
2. A board cell contains A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins if {line_length} of their pieces form a straight line (horizontal,
↪  vertical, or diagonal) and every consecutive pair of points in that line is adjacent
↪  under rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Wording 5.

```
You will receive the rules of a new board game. Then you will be given one final state of
↪  the board from that game. Determine the outcome currently shown.

RULES:
1. The board is a {board_size}x{board_size} point grid using coordinates (x,y), where 0 <=
↪  x,y < {board_size}.
2. A board cell contains A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
```

```
4. A win occurs for a player if they have {line_length} pieces in a straight line
↪  (horizontal, vertical, or diagonal) and each consecutive pair of points in that line is
↪  adjacent under rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Wording 6.

```
You will study a new board game and then receive one final board state from that game.
↪  Decide which current game outcome applies.

RULES:
1. The board is a {board_size}x{board_size} grid of points, each with coordinates (x,y),
↪  where 0 <= x,y < {board_size}.
2. A board cell contains A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins if they have {line_length} pieces arranged in a straight line (horizontal,
↪  vertical, or diagonal) and each consecutive pair of points in that line is adjacent
↪  under rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Lexical Category Inference, easy level

Wording 1.

```
You will receive a list made up of four words. Determine whether the list is correct
↪  according to these rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. The meaning of each category is given in the category definitions below.
3. A correct list has exactly four items separated by commas.
4. For a list to be correct, item one must belong to Category 1, item two must belong to
↪  Category 2, and this pattern continues in order.
5. When a category says "at least one", an item may match any of the listed descriptions.
↪  When it says "all", an item must match every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Wording 2.

```
A list of four words will be presented to you. Judge whether it is correct using the rules
↪  below.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category means what is specified in the category definitions below.
3. A correct list consists of exactly four comma-separated items.
4. In a correct list, the items correspond in order: the first item is in Category 1, the
↪  second item is in Category 2, and so on.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Wording 3.

```
You will be given four words as a list. Decide if that list is correct by applying the
↪  following rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. The category definitions below state the meaning of each category.
3. To be correct, a list must include exactly four items separated by commas.
4. To be correct, the first item must be a member of Category 1, the second item must be a
↪  member of Category 2, and so forth.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Wording 4.

```
You will see a four-word list. Assess whether this list is correct under the rules that
↪  follow.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category has the meaning provided in the category definitions below.
3. A list is correct only if it contains exactly four comma-separated items.
4. A correct list places a Category 1 member first, a Category 2 member second, and
↪  continues in that order.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}
```

```
List: {candidate_block}
Answer:
```

Wording 5.

```
You will be shown four words in list form. Decide whether the list qualifies as correct
↪  according to these rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. The definitions below give the meaning for each category.
3. Exactly four comma-separated items are required for a correct list.
4. In a correct list, the first, second, and later items must respectively be members of
↪  Category 1, Category 2, and the following categories in sequence.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Wording 6.

```
A list containing four words will be provided. Determine whether it is correct based on the
↪  rules listed here.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category's meaning is the one shown in the category definitions below.
3. For a list to be correct, it must contain exactly four items, with commas separating them.
4. For a list to be correct, its first item must belong to Category 1, its second item must
↪  belong to Category 2, and the remaining items must follow the same order.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

D Term lists

The term list of each complex task at the easy level, generated and checked as in Sec. 5.3, as words before their expansion to token ids. Tapatan's list is made from its single prompt, shared by all levels.

Task (easy level)	Terms
Set Game	set, sets, Set, Sets, card, cards, Card, Cards, board, boards, Board, Boards, select, selects, Select, Selects, identify, identifies, Identify, Identifies, find, finds, Find, Finds, choose, chooses, Choose, Chooses, match, matches, Match, Matches, matching, Matching, valid, Valid, invalid, Invalid, attribute, attributes, Attribute, Attributes, animal, animals, Animal, Animals, biome, biomes, Biome, Biomes, same, Same, different, Different, all, All, triple, triples, Triple, Triples, triad, triads, Triad, Triads, combination, combinations, Combination, Combinations, group, groups, Group, Groups, game, games, Game, Games, gameset, gamesets, Gameset, Gamesets, rule, rules, Rule, Rules, constraint, constraints, Constraint, Constraints, value, values, Value, Values
Operator Function	arithmetic, Arithmetic, operator, operators, Operator, Operators, function, functions, Function, Functions, calculation, calculations, Calculation, Calculations, compute, Compute, calculate, Calculate, evaluate, Evaluate, solve, Solve, multiplication, multiplications, Multiplication, Multiplications, multiply, Multiply, product, products, Product, Products, addition, additions, Addition, Additions, add, Add, sum, sums, Sum, Sums, integer, integers, Integer, Integers, input, inputs, Input, Inputs, result, results, Result, Results, expression, expressions, Expression, Expressions, composition, compositions, Composition, Compositions
Noun Class Agreement	grammar, Grammar, grammars, Grammars, grammaticality, Grammaticality, grammatical, Grammatical, ungrammatical, Ungrammatical, syntax, Syntax, sentence, Sentence, sentences, Sentences, phrase, Phrase, phrases, Phrases, noun, Noun, nouns, Nouns, determiner, Determiner, determiners, Determiners, class, Class, classes, Classes, agreement, Agreement, match, Match, matches, Matches, matching, Matching, mismatch, Mismatch, mismatches, Mismatches, template, Template, templates, Templates, validation, Validation, classification, Classification, decision, Decision, decisions, Decisions, binary, Binary, well-formedness, Wellformedness, acceptability, Acceptability
Tapatan	board, boards, Board, Boards, game, games, Game, Games, grid, grids, Grid, Grids, cell, cells, Cell, Cells, point, points, Point, Points, coordinate, coordinates, Coordinate, Coordinates, piece, pieces, Piece, Pieces, player, players, Player, Players, state, states, State, States, outcome, outcomes, Outcome, Outcomes, result, results, Result, Results, winner, winners, Winner, Winners, win, wins, Win, Wins, winning, Winning, line, lines, Line, Lines, alignment, alignments, Alignment, Alignments, sequence, sequences, Sequence, Sequences, straight, Straight, horizontal, Horizontal, vertical, Vertical, diagonal, diagonals, Diagonal, Diagonals, orthogonal, Orthogonal, neighbor, neighbors, Neighbor, Neighbors, adjacency, adjacencies, Adjacency, Adjacencies, adjacent, Adjacent, connection, connections, Connection, Connections, connectivity, Connectivity, parity, parities, Parity, Parities, even, Even, empty, Empty, validity, Validity, valid, Valid, classification, classifications, Classification, Classifications, classify, Classify, determine, Determine, detect, Detect, evaluate, Evaluate, identify, Identify, label, labels, Label, Labels
Lexical Category Inference	validation, Validation, validate, Validate, verification, Verification, assessment, Assessment, decision, Decision, classification, Classification, classify, Classify, categorization, Categorization, categorize, Categorize, category, Category, categories, Categories, membership, Membership, member, Member, members, Members, list, List, lists, Lists, item, Item, items, Items, word, Word, words, Words, order, Order, ordering, Ordering, sequence, Sequence, position, Position, positions, Positions, alignment, Alignment, matching, Matching, match, Match, criteria, Criteria, criterion, Criterion, rule, Rule, rules, Rules, constraint, Constraint, constraints, Constraints, definition, Definition, definitions, Definitions, correctness, Correctness, incorrectness, Incorrectness, satisfaction, Satisfaction, validity, Validity

Decision log

2026-10-27 Scope: test the lexical-task-head method of Yang et al. (2026) on complex tasks and compare the per-head logit lens with Head-JLens. New title.

2026-10-27 Tasks: the 14 simple tasks shared by Yang et al. (2026) and Davidson et al. (2025); the five tasks of Fu et al. (2026).

2026-10-27 No main condition: rule-based and example-based prompts are treated alike (replacing the earlier choice of rules as main and examples as secondary).

2026-10-27 Method: lexical task heads, with Logit-head and J-head readouts. Function vectors, function-specific heads and the hidden-state difference between prompts are not used for now.

2026-10-27 At least six rule wordings per complex task and difficulty level.

2026-10-27 Term lists are task-specific, not item-specific; one per task, or per difficulty level where the rules change.

2026-10-27 Answer format as in the source studies.

2026-10-27 Model: Llama-3.1-8B-Instruct, the main model of Yang et al. (2026) and the only one with released Head-JLens matrices.

2026-10-27 Rule wordings and term lists for the complex tasks are generated with GPT-5.5 through ChatGPT and checked by hand.

2026-10-27 Accuracy is computed with each source paper's evaluation.

2026-10-27 Prompts on which the model does not answer straight away are left out; other ways of handling them are considered after E0.

2026-10-27 After an early E0 finding (no direct answers on Set Game and Operator Function), these tasks are run in reasoning mode (Sec. 5.3).

2026-10-27 Tapatan is run in reasoning mode as well: in E0 none of its 300 items is answered straight away, under any prompt.

2026-10-27 Complex tasks: the easy level of each task only.

2026-10-27 Rule wordings and term lists are generated with GPT-5.5 through the Azure OpenAI API, replacing ChatGPT; six wordings per original rule prompt. Both were checked by hand and approved.

2026-10-27 Complex tasks are decoded greedily, and use 300 test items per task drawn with the sampler and seed of Fu et al. (2026) (all 600 for Lexical Category Inference), so that the causal check can leave the patched items out of the mean.

2026-10-27 Reasoning mode: 100 items per task, style and wording; the causal check patches 10 items with $\alpha \in \{0, 1, 4, 10\}$.

2026-10-28 Complex tasks follow the simple-task rules: rule-style heads from the most accurate rule wording (replacing the pooling over the seven wordings), and in the causal check one mean and one patch per rule wording; the control mean is the control task's mean under its original rules prompt. The causal check of the complex tasks patches the rule-style head sets only. The second check is adapted to them (original rules prompts answered wrongly, no control). Reasoning mode uses 300 items, as the other complex tasks (replacing 100); the first 100 are the earlier sample, whose replies are reused. E2 was rerun.

2026-10-29 Reasoning mode: each distinct prompt is answered once, and its reply scored for every item that shares it (Tapatan's 300 items repeat 46 boards, and a repeated prompt could get a different greedy reply in another batch).

2026-10-29 Answer evaluation (Sec. 5.5), after Hua et al. (2025): every E0 reply is generated once with longer limits and saved; each is scored with the source evaluation and with a GPT-5.5 judge, and every disagreement is given a reason. The judge sees every reply, not only those the strict score rejects. Head identification and the causal checks keep the strict score until this evaluation is done.

2026-10-29 One generation scheme per group of tasks (Sec. 5.5). Simple tasks: a reply ends at `;`, at a newline once the reply has begun (a leading newline no longer ends it), at the end of sequence, or at 32 new tokens. Complex tasks: a reply ends at the end of sequence or at 1,024 new tokens, for all five tasks; this replaces the limit of 6,000 in reasoning mode and the stop once the scored answer is complete. The saved outputs are generated again under this scheme. Head identification and the causal checks in reasoning mode were run with the earlier limit of 6,000 tokens and the stop at the answer.

2026-10-29 Complex tasks: no distinction between direct and reasoning mode. All five are generated to the end of sequence or 1,024 tokens and scored on the full reply, by the evaluation of Fu et al. (2026) (strict; Tapatan, which has no scorer in their code, by its first line) and by the judge.

2026-10-30 Tapatan's strict score reads the first label anywhere in the reply, as in the first run of E0, replacing the first line: the model reasons before its label, so the first line is the label in 1 of 3,000 replies.

2026-10-29 Heads are read at the last prompt token only; the pre-answer position is dropped, with its results. Which prompts form P^+ for the complex tasks is decided later. Causal checks: the patch is applied at the last prompt position only, for every task, and every task uses the same 15 values of α.

2026-10-30 Causal checks of the complex tasks: $\alpha \in \{-10, -6, -4, -2, 0, 1, 2, 4, 6, 8, 10\}$, the even values of Yang et al. (2026) and 1, to bound the cost of 1,024-token replies; the simple tasks keep all 15.

2026-10-30 Complex tasks: a flexible score next to the strict one (Sec. 5.5), the answer the strict score looks for in any notation, scored by the strict score; Tapatan's is its strict score. It is reported for E0; head identification and the causal checks keep the strict score. Under the flexible score, Operator Function's rule wordings would all tie at 1.00, and its second causal check would have no items.

2026-10-30 No hand-labelled validation of the judge; its use rests on the human check of Hua et al. (2025).

2026-10-30 Head identification and the causal checks use the flexible score, replacing the first-token score (simple tasks) and the strict score (complex tasks): P^+ is the prompts answered correctly under it, with no condition that the answer comes first, for both families; the identification wordings, the head sets, the means, the patched items and the controls are chosen by it. Readings and means are taken from the prompts of the saved outputs. In the causal checks the reply is generated as the saved outputs and scored by the flexible score; for the simple tasks this replaces the first token, which is kept as a second score. Both complex-task checks patch 100 items on every task, replacing 10 on Set Game, Operator Function and Tapatan; Operator Function has no second check, as no item is answered wrongly under its original rules prompt. E1 and E2 are rerun.

2026-10-31 The second causal check of the complex tasks gets the control mean of the first check (the control task's mean under its original rules prompt, at the same heads and α). Without it, zero-ablation already fixed part of the failures of Set Game and Tapatan, and on the two-label tasks the gains came with one label for every item. The simple tasks' second check stays without a control, as in the released script of Yang et al. (2026).

2026-11-03 The project is split into two documents: `proposal-simple`, the identification method on the simple tasks (E0 for the simple tasks, E1, E1b), and `proposal-complex`, the complex tasks (E0 for the complex tasks, E2). Work continues on the simple tasks for now. The readout hypothesis is stated once per document, with its criteria unchanged.